

Human Activity Recognition in Smart Home Environments

Ambient Intelligence and Domotics

Master Degree: Artificial Intelligence for Science and Technology

Gabriele Civitarese

EveryWare Lab, Department of Computer Science

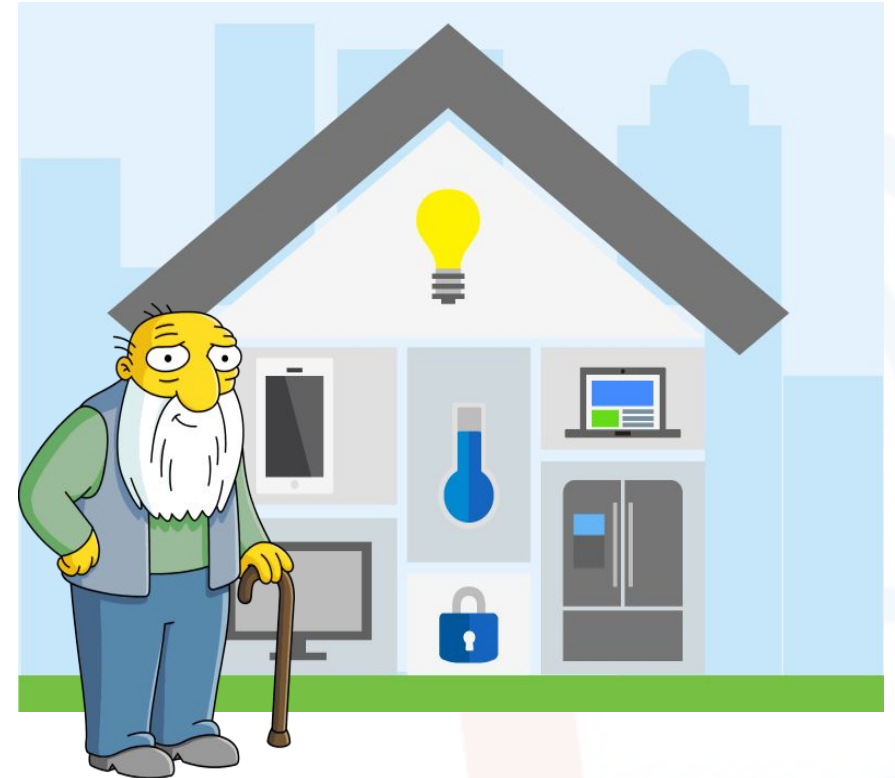
Università degli Studi di Milano

Copyright

These slides are from the teachers of this course. All the material is subject to copyright and cannot be redistributed without consent of the copyright holder. The same holds for audio and video-recordings of the classes of this course.

Smart-homes

- Smart-home environments equipped with sensors can continuously monitor the high-level activities of the residents
- This is particularly important for ambient assisted living applications
 - e.g., monitoring elderly subjects to detect early symptoms of cognitive decline, emergency monitoring



Sensing in smart-homes for HAR

Binary environmental sensors

- The most common environmental sensors used for smart-home HAR are called **binary sensors**
- They indeed can generate only two states: ON and OFF
- Environmental sensors are used to monitor the interaction of the resident with the home environment (**dense sensing**)
 - opening/closing doors, drawers, fridge, ...
 - presence in certain home locations
 - usage of appliances
 - sitting on chairs
 - lying on bed
 - etcetera



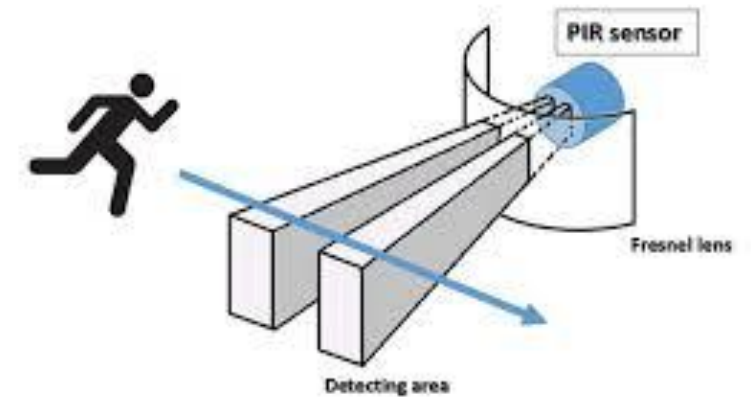
Magnetic Sensors

- Sensors that can detect opening and closing of doors/drawers using a magnet
 - when the magnet is close to the sensor, the state is OFF (closed)
 - otherwise it is ON (open)
- It is possible to monitor many home components
 - e.g., kitchen drawers, medicine drawers, doors, fridge, closets, etcetera



Passive InfraRed (PIR)

- PIR sensors can detect human motions/proximity in specific areas
- They can be used to monitor the presence of the user in specific rooms or to monitor their movements in the home
- They can only sense ***motion***! If the user is still, this sensor is not capable of revealing the presence



Pressure mat sensors

Pressure mats can be used to detect when someone is sitting on a chair, or when he stands up



Sleep monitoring sensors

- Smart mat sensors can be also used to unobtrusively monitor the sleep quality
- What can be captured:
 - sleep phases (e.g., light, REM)
 - snoring detection
 - heart rate
 - ...

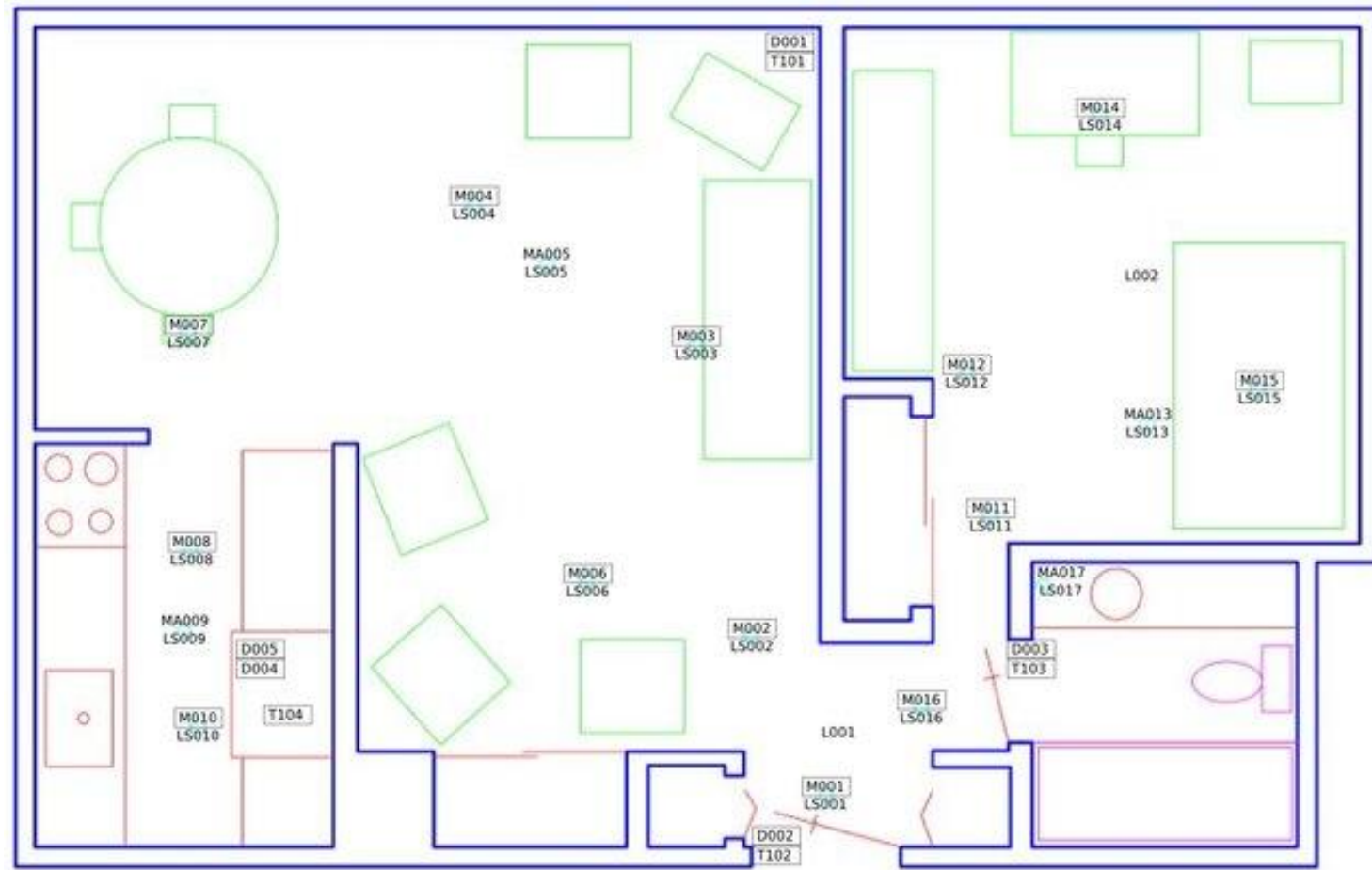


Smart Plugs

- As discussed in the smart energy management lesson, plug sensors can detect the usage of appliances
- While these sensors may reveal power usage in details, they can also be treated as binary sensors
- The idea is to use thresholds to monitor whether appliances are turned ON and OFF



A typical smart-home sensor map

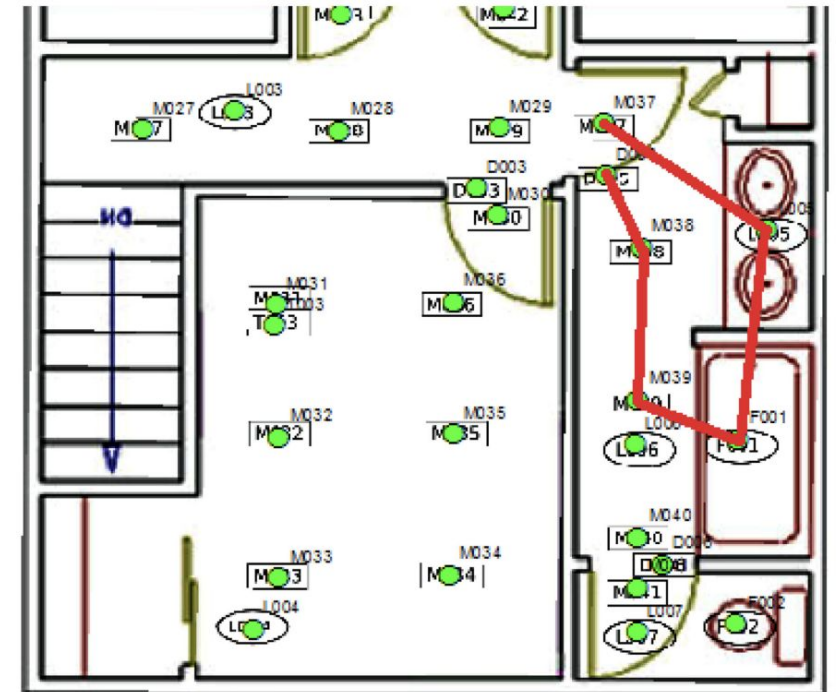


Binary sensors log

2010-11-04	05:40:51.303739	M004	ON
2010-11-04	05:40:52.342105	M005	OFF
2010-11-04	05:40:57.176409	M007	OFF
2010-11-04	05:40:57.941486	M004	OFF
2010-11-04	05:43:24.021475	M004	ON
2010-11-04	05:43:26.273181	M004	OFF
2010-11-04	05:43:26.345503	M007	ON
2010-11-04	05:43:26.793102	M004	ON
2010-11-04	05:43:27.195347	M007	OFF
2010-11-04	05:43:27.787437	M007	ON
2010-11-04	05:43:29.711796	M005	ON
2010-11-04	05:43:30.279021	M004	OFF
2010-11-04	05:43:45.7324	M003	ON
2010-11-04	05:43:52.044085	M003	OFF
2010-11-04	05:43:53.185335	M002	ON
2010-11-04	05:43:53.253809	M003	ON
2010-11-04	05:43:59.493281	M002	OFF
2010-11-04	05:44:04.048766	M003	OFF
2010-11-04	05:44:06.14204	M003	ON
2010-11-04	05:44:11.229146	M003	OFF

Indoor localization

- Understanding the position of the residents and their trajectories is a crucial context information
- Binary sensors are in fixed positions and they may reveal the presence of the resident in a specific position
- However:
 - they are not everywhere in the home
 - they can not identify the resident
- Indoor localization technologies can hence be used to provide richer context information!
- **Drawback:** the resident has to wear a mobile/wearable device



Use of mobile/wearables

- While only relying on mobile/wearable devices it is not possible to determine complex high-level activities, it is possible to combine them with environmental sensors to obtain more information
 - e.g., combining smartwatch gestures with environmental sensor data, using the posture information to understand if the user is sitting or standing, etcetera
- There are also specific applications where wearable plays a major role
 - e.g., fall detection



Problems of audio/video based sensing

- Audio and video solutions may be used as additional context sources in smart homes
- However, cameras and microphones may be perceived as too privacy intrusive (especially by elderly subjects)
 - for this reason we will not consider these solutions in this course



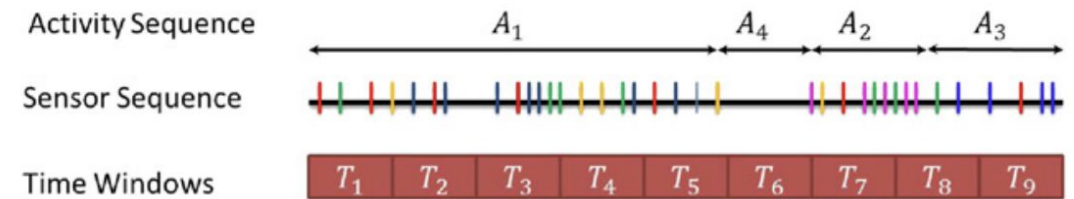
Segmentation

Segmentation and feature extraction

- Binary environmental sensors have different characteristics with respect to the sensors of mobile/wearable devices
- In the following, we will see some approaches to pre-process smart-home sensors data to obtain windows and feature vectors

The problem of time-based windowing

- Classic approaches based on fixed time sliding windows are simple to implement, efficient, and good for sensors generating continuous values (e.g., accelerometer)
- However, they may not always be a good choice for binary environmental sensors
 - there is no fixed sampling rate, hence the distribution of events in windows is not consistent
 - many windows may not have sensor events when the subject is not performing activities



Event-based windowing

- Segment the stream into windows containing a fixed number of sensor events
 - The time interval covered by each window is variable
 - The number of events is a parameter requiring proper tuning
- Why is it better than time-based windowing?
 - while performing activities, sensor events will be dense
 - during silent periods sensor events will be sparse (or absent)
 - hence, with this approach we will avoid unrepresentative time windows

Handcrafted Feature Extraction

Basic feature extraction

- Given a window of the latest k sensor events, it is needed to transform it into a feature vector capturing its information content
- Basic approach: build a fixed dimensional feature vector with information like:
 - the timestamp of the first sensor event
 - the timestamp of the last sensor event
 - the temporal span of the window
 - *activation count*: for sensor event type, the number of times it was activated during the window

Problems of the basic approach

- In event-based segmentation, there may be significant time lags between consecutive sensor events
 - if two sensor events are significantly temporally distant, it makes poor sense to consider them together as part of the same activity!
 - hence, it is not correct to give equal importance to each event
- While transitioning from an activity to another, the same window may contain events related to two different activities
- Event-based windowing should be considered in conjunction with appropriate strategies to capture time and semantic dependencies between events

How to deal with this problems?

- When using event-based segmentation, many methods classify the activity associated with the latest sensor event in the window, leveraging the past $k-1$ event as context
 - each time a sensor event occurs in the smart-home, a window with this event and the previous $k-1$ events is created
 - the ground truth of the window is the activity performed by the subject on the ***last event***
 - this also improves real-time recognition (i.e., it's a sliding window shifting by a single event)
 - however, we need some tricks to obtain meaningful features from such strategy

Time-dependency features

- We want to classify the current activity based on the last sensor event of the window...
 - ... but only the sensor events temporally close to it should be relevant!
- The activation count is transformed into a **discounted count**:
 - For each sensor j , each activation of this sensors is weighted by the time distance (considering a temporal decay rate χ) with respect to the latest sensor event in the window:

$$C(i, j) = \exp(-\chi (t_i - t_k))$$

↑
time contribution of a
sensor j compared to the
latest activation of the
window given by sensor i
at time t_i

↑
temporal
decay
rate

↑
time of the
last event in
the window

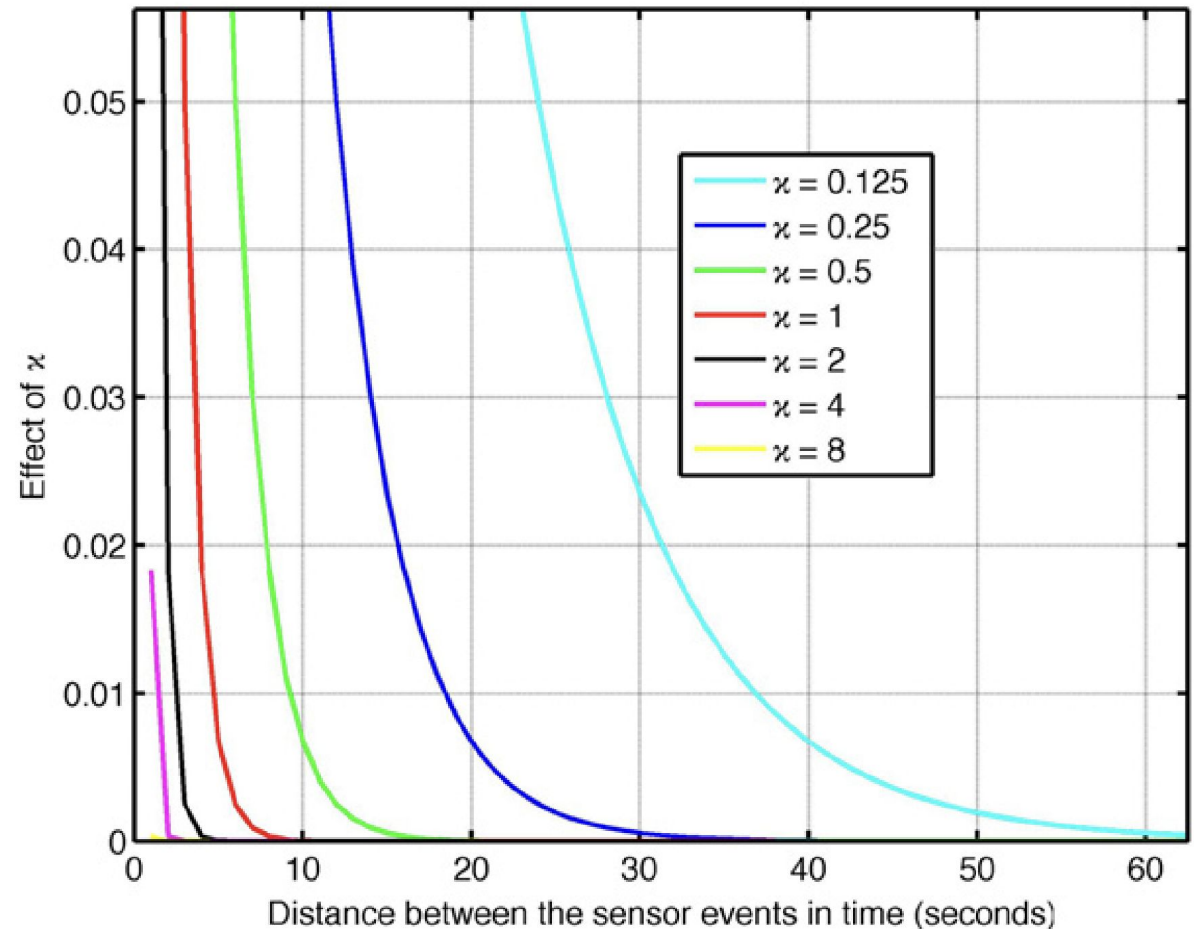
↑
time of
the event of
type j

Time-dependency Features (cont)

- Consider a sensor-based window $w = \{S_{i-k}, \dots, S_{i-1}, S_i\}$
 - S_i is the latest sensor event!
- Create a feature for each type of sensor event j (e.g., FridgeEvent, CookerEvent) as follows:
 - For each activation S_a of a sensor type j in w , compute $C(S_a, S_i)$
 - The value of the feature is the sum of all the computed $C(S_a, S_i)$

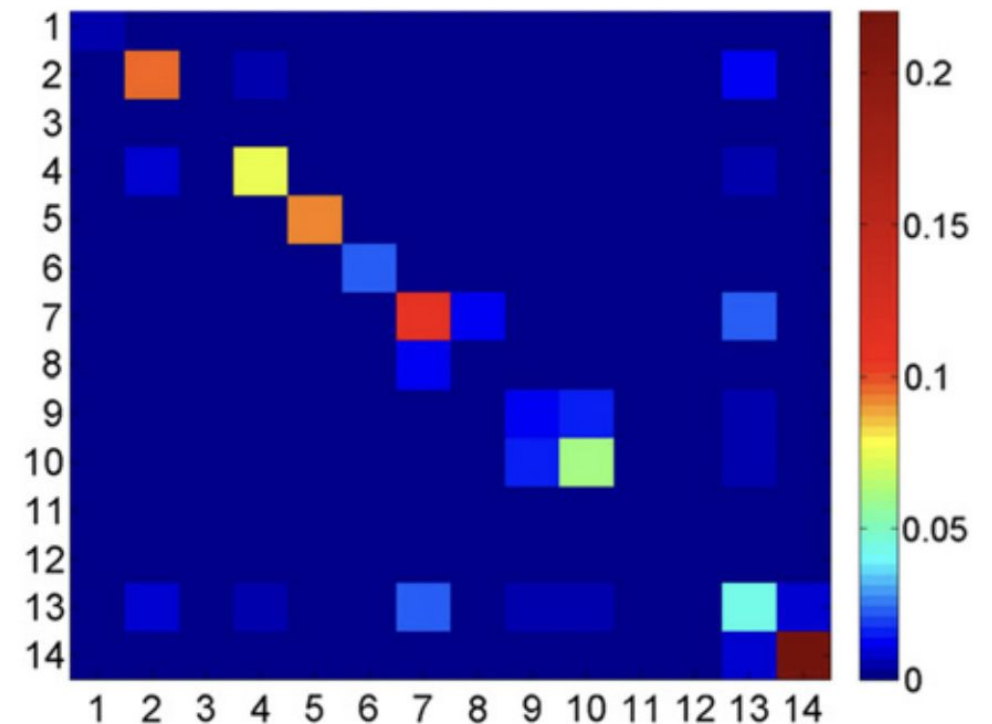
Effect of χ on the time contribution

- If the value of χ is too low, temporally distant sensor events will impact the recognition rate
- If χ it is too high, even temporarily close sensor events may be considered unimportant for classification
- Hence, the goal is to find a trade-off (empirically)



Sensor-dependency Features

- Another problem: during transitions between two different activities, a window may contain temporally close sensor events poorly related between them
- A possible solution is to compute a Mutual Information matrix:
 - it encodes correlations between pairs of random variables
 - in our case, a random variable is a sensor event emitting “ON” and “OFF”
 - a pair of random variables are sensor events occurring consecutively in the dataset



Sensor-Dependency (cont)

- From the training set, Mutual Information is computed for each pair of sensor events type S_i and S_j as the chance of these events occurring consecutively in the stream:

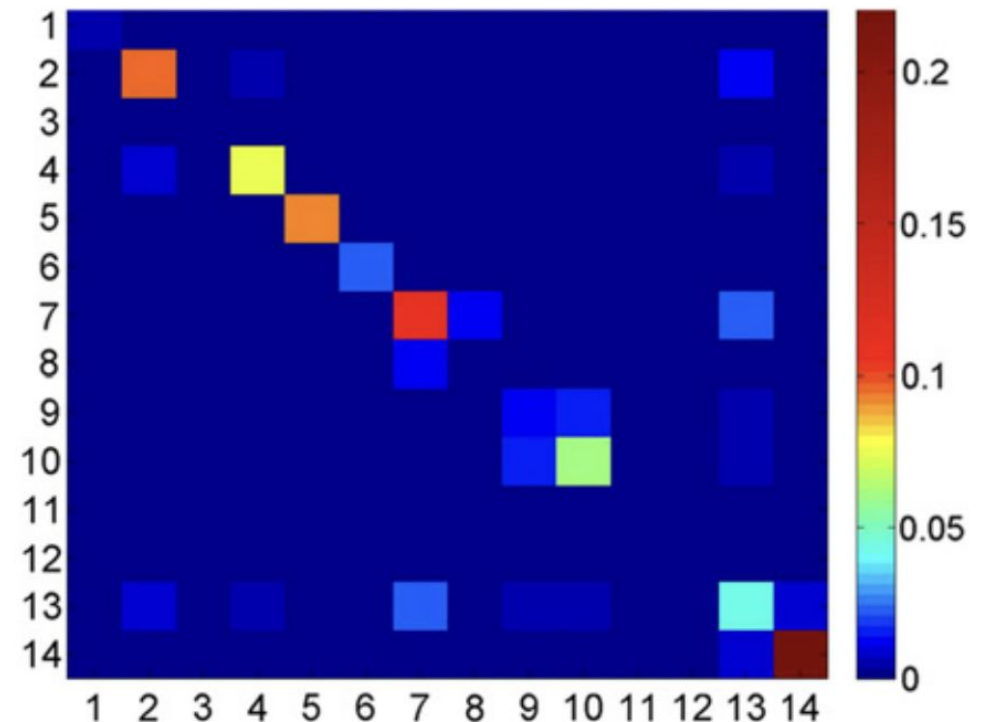
$$MI(i, j) = \frac{1}{N} \sum_{k=1}^{N-1} \delta(s_k, S_i) \delta(s_{k+1}, S_j)$$

$\uparrow$ $\uparrow$
k-th event in the training set (k+1)-th event in the training set

$$\delta(s_k, S_i) = \begin{cases} 0 & \text{if } s_k \neq S_i \\ 1 & \text{if } s_k = S_i. \end{cases}$$

Using the Mutual Information matrix

- Once the MI matrix is ready, it is possible to use it to associate weight the activation count
 - similarly to *time dependency*, considering the mutual information w.r.t. the last event in the window
 - **drawback:** the matrix may overfit the specific training set being used
 - it is possible to combine *sensor dependency* and *time dependency* together!



Past activities

- Besides considering the information in the current window, the previous activities recognized by the model may add hints to recognize the current activity
 - for instance, it is likely that the resident *washes dishes* after *eating*
- Hence, the output of the classifier in previous windows may be used to add additional features about past activities
- Not trivial! Past classifications may be incorrect, so their confidence should be used

Automatic Feature Extraction

Back to time windows again

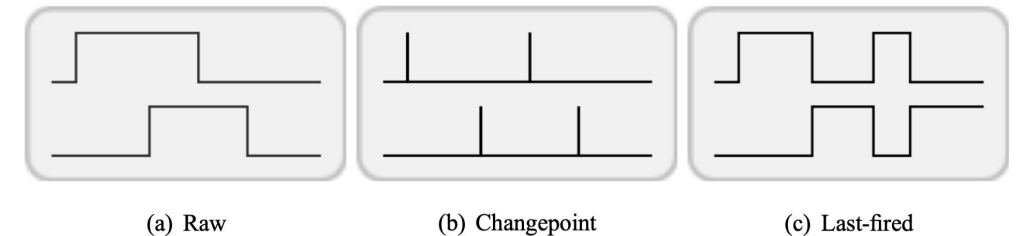
- In automatic feature extraction, we want deep learning models to generate embeddings from raw (pre-processed) time series
- Usually, these models require fixed size input and time-based segmentation is more appropriate
- In the following, we will understand how to encode fixed-size time windows of binary sensor events so that they are processed by deep learning models

How to encode sensor events as time series?

Goal: encode the sensor event activations so that as a binary time series.

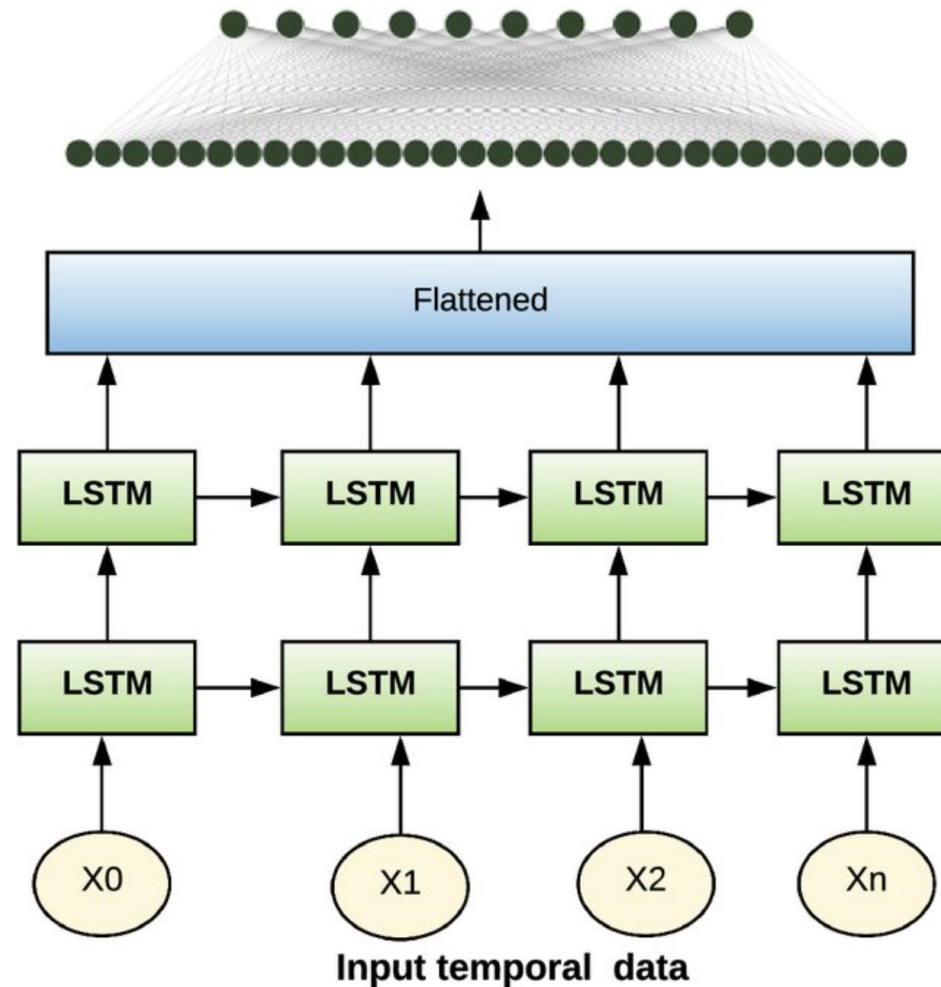
Different encoding approaches:

- a) **Raw data:** at each instant in the time window (e.g., each second), encode 1 if a sensor event is active, 0 otherwise
- b) **Change point:** encode 1 only in the time instants where the sensor event status changes (e.g., from ON to OFF)
- c) **Last-fired:** at each time instant, only the latest sensor event is encoded with 1

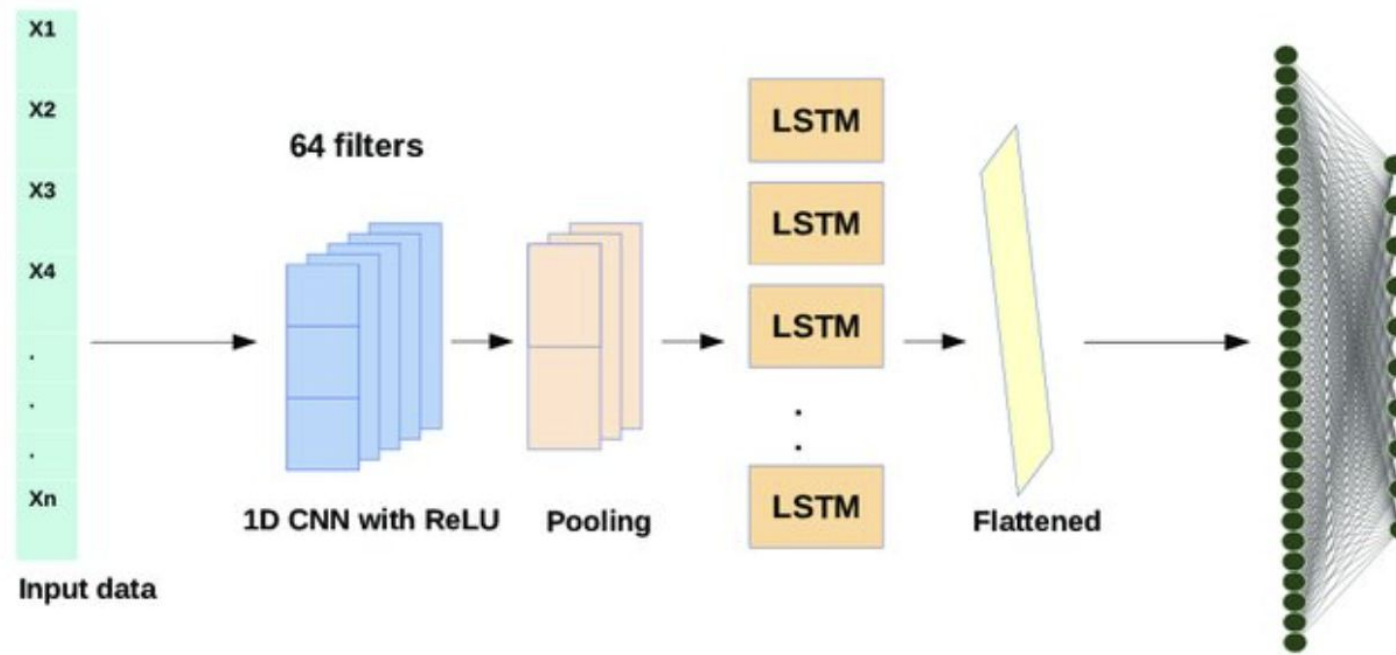


Note that it is possible to combine multiple encodings, by creating more than one time series for each sensor.

Using LSTMs

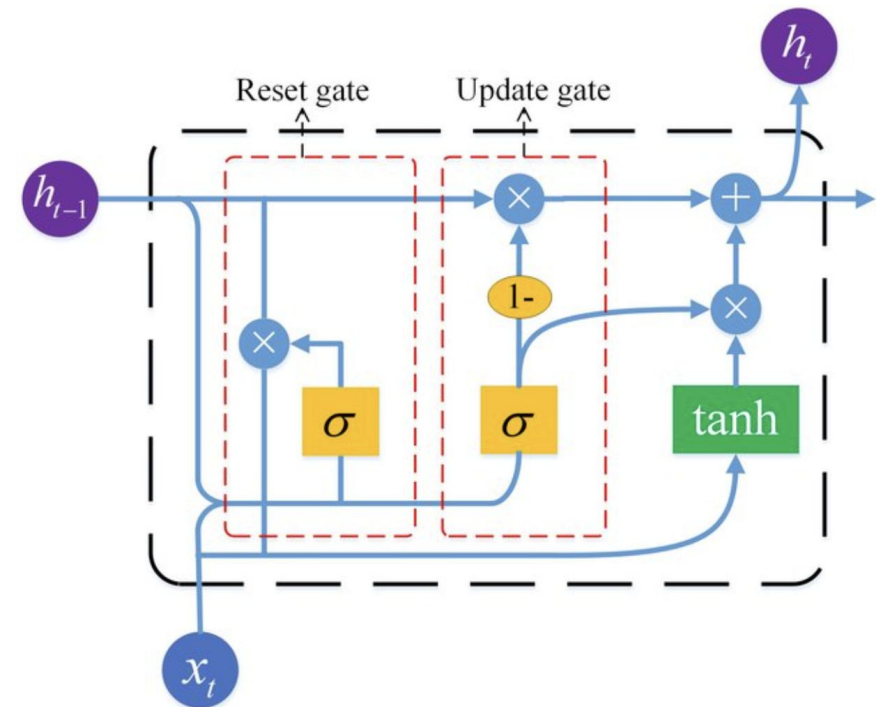


LSTM-CNN

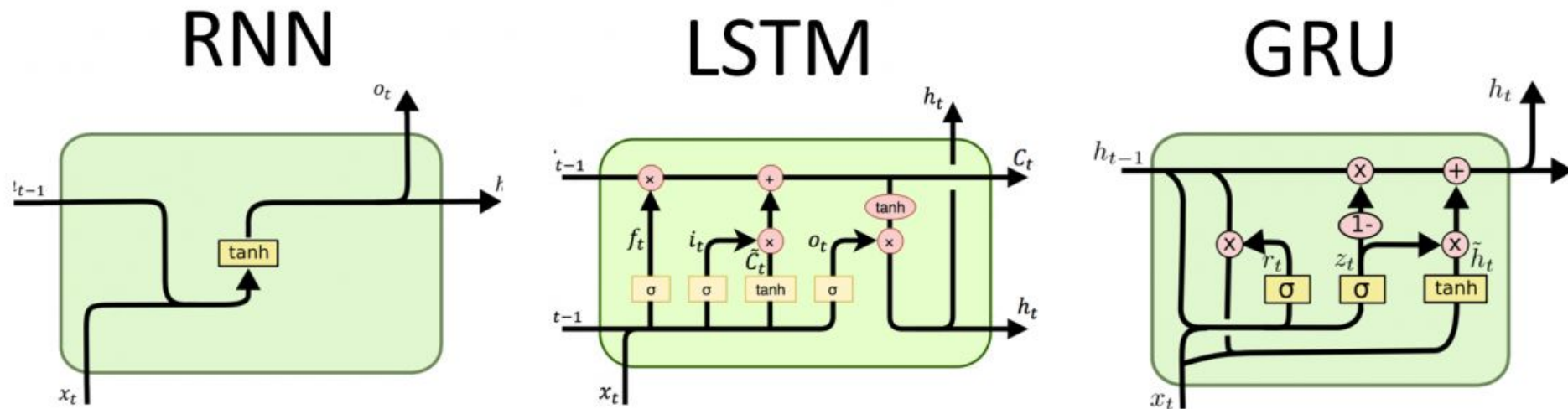


Gated Recurrent Unit (GRU)

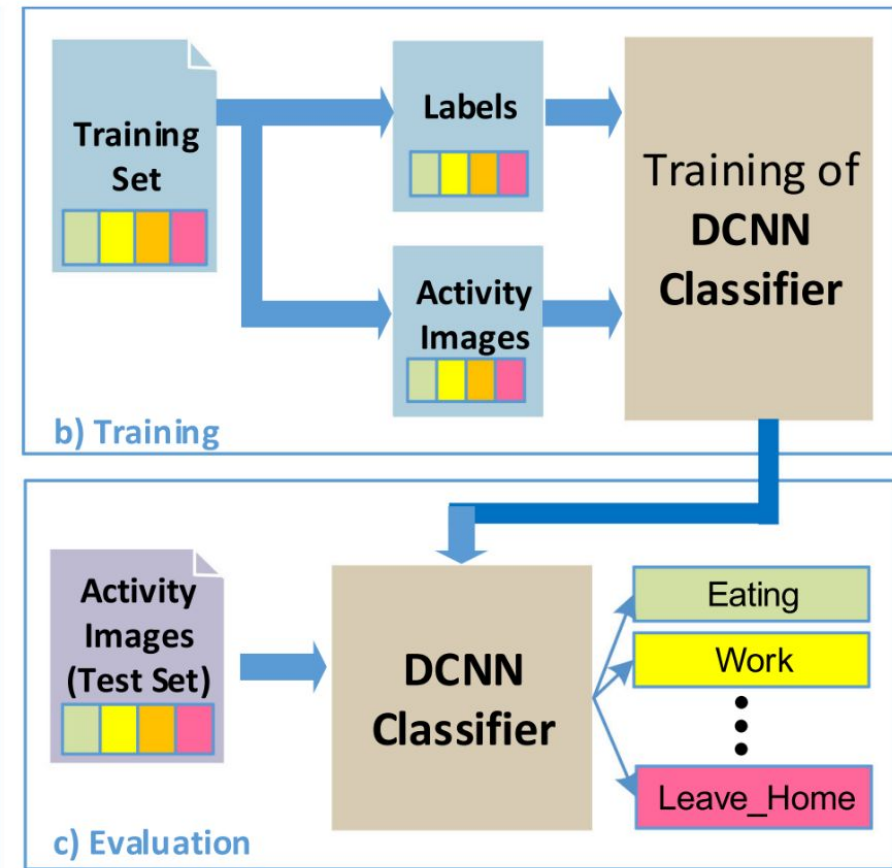
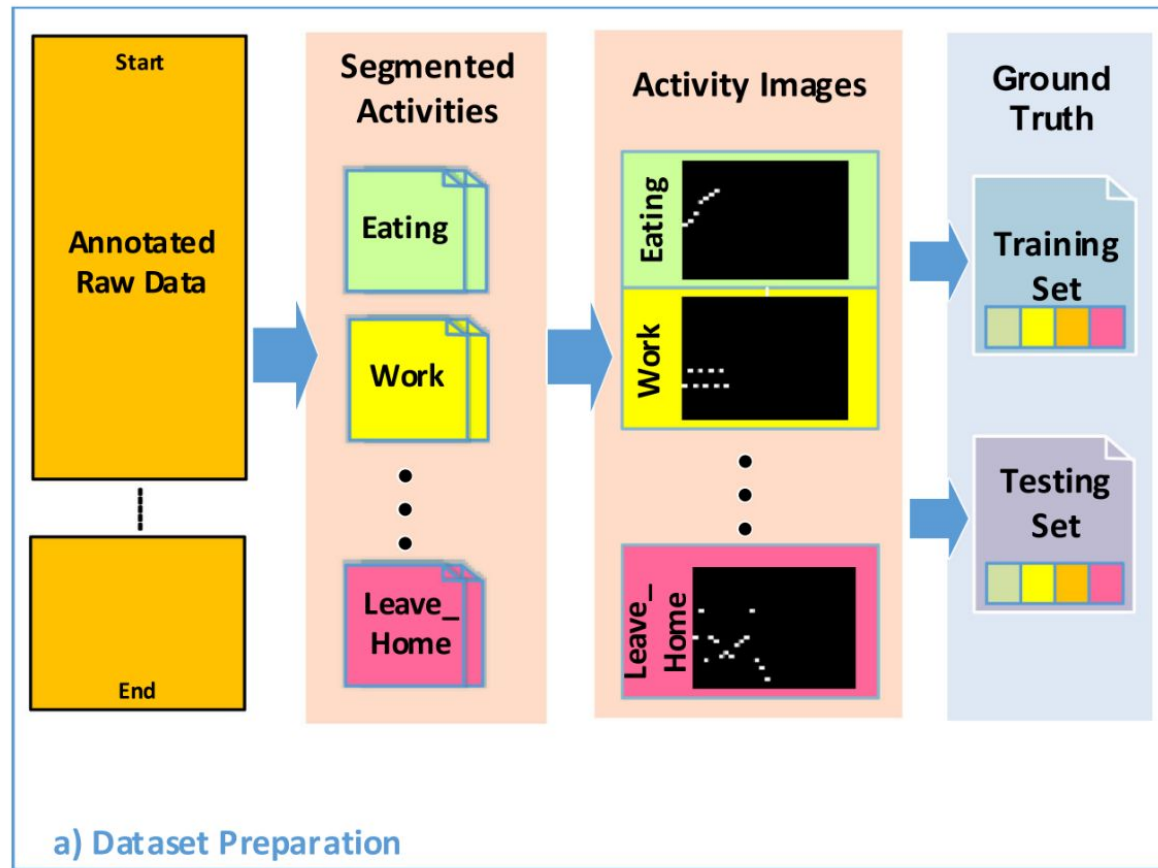
- With small datasets, LSTM may lead to overfitting
- Hence, sometimes in this domain GRU are preferred
- Light version of LSTM units with only two gates and one memory (the long-term one, that also corresponds with the cell's output):
 - **reset gate**: it determines the % of long-term memory to forget based on the input
 - **update gate**: it determines how to combine the input and the long-term memory to generate the output
- Faster to train and less prone to overfitting in small datasets
- Not adequate for long sequences



RNN vs LSTM vs GRU



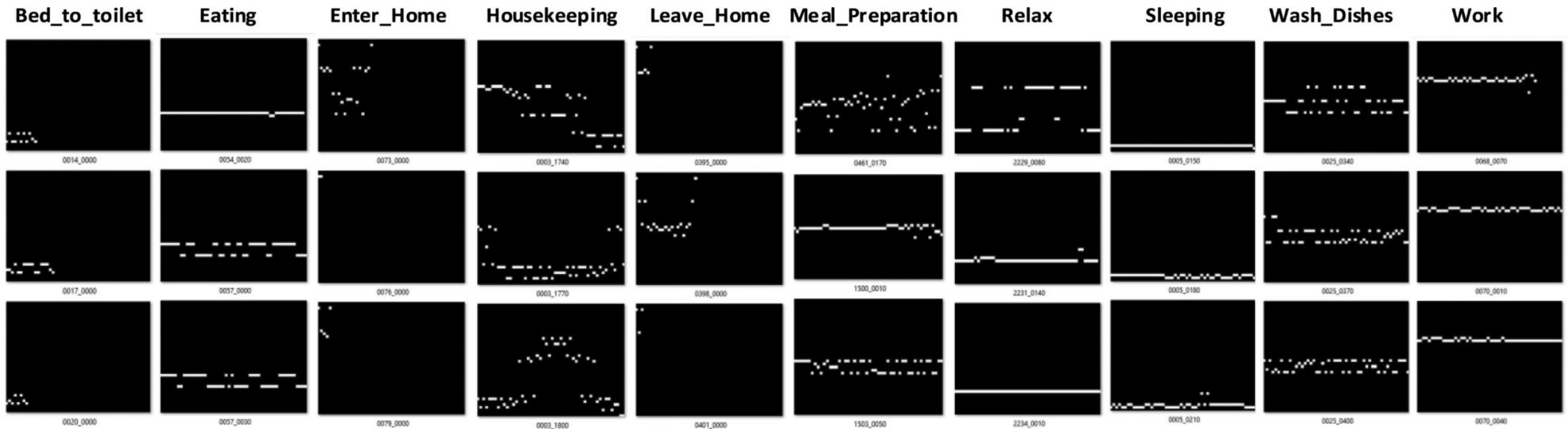
CNNs with image representation



About image representations for CNNs

- How to compute the activity images?
- Main idea:
 - binary matrix (i.e., pixels may be black or white)
 - the x axis encode time
 - it spans length of the window
 - a granularity should be defined (e.g., each cell is 1 second)
 - each row encodes a specific sensor event
 - white pixels represents the time instants where a sensor event was active

Image representation of sensor data



[Gochoo, M., Tan, T. H., Huang, S. C., Liu, S. H., & Alnajjar, F. S. (2017, November). DCNN-based elderly activity recognition using binary sensors. In *2017 international conference on electrical and computing technologies and applications (ICECTA)* (pp. 1-5). IEEE.]

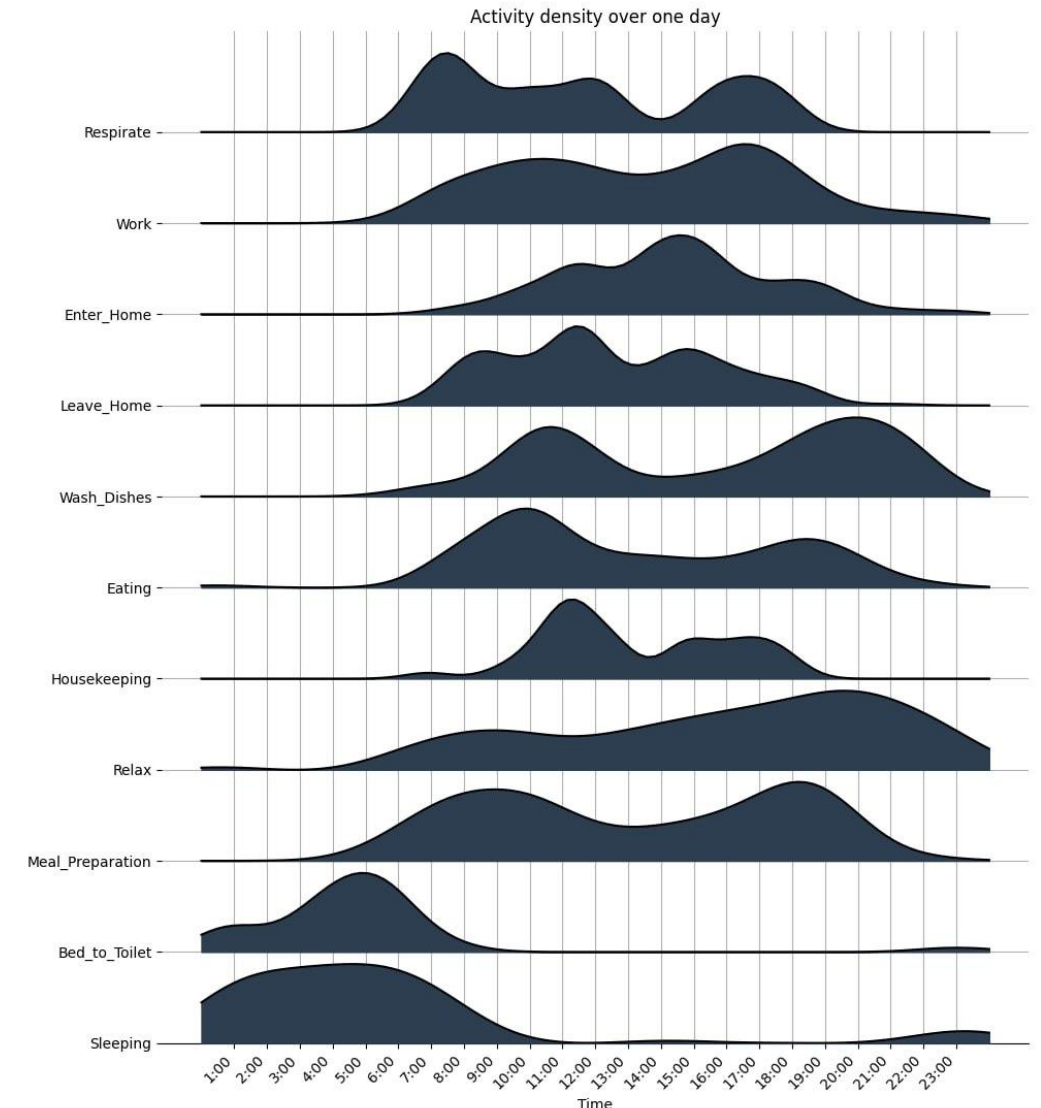
Recognizing Activities to Detect Behavioral Changes

Detecting long-term behavioral changes

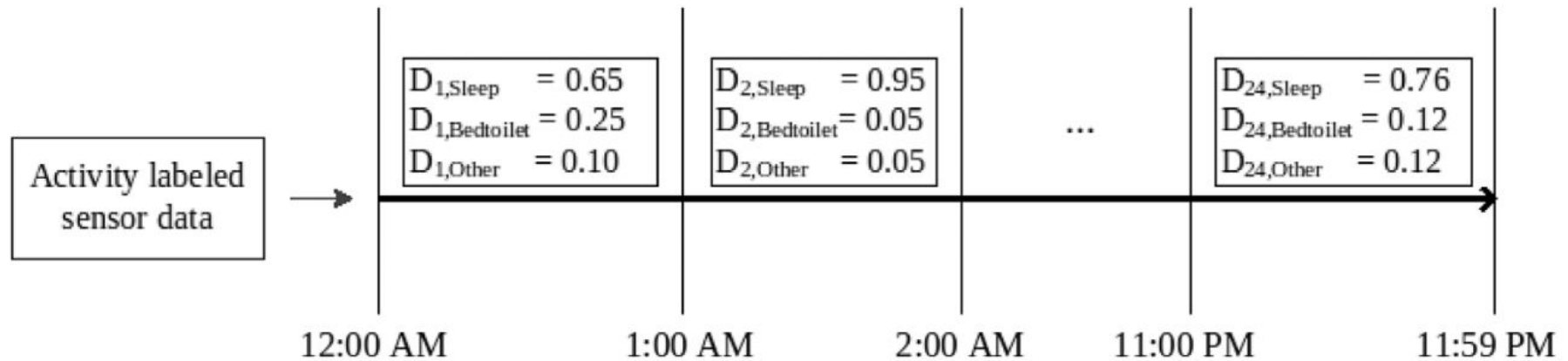
- ADLs recognition in smart homes has applications in healthcare
- Indeed, by analyzing the behavior of a subject on the long-term, it is possible to detect significant changes in activity execution
 - these may be early indicators of problems like cognitive decline

Activity Curves

- An **activity curve** is a model representing an individual's generalized activity routine.
- Useful to monitor activity routines at different time granularities (e.g., a daily, weekly, monthly routines)
- These curves show behavioral characteristics based on the timing of recognized activities
- How to build them?
 - segment an observation period into equal-size consecutive time windows and define probability distributions over activities



Activity Distribution

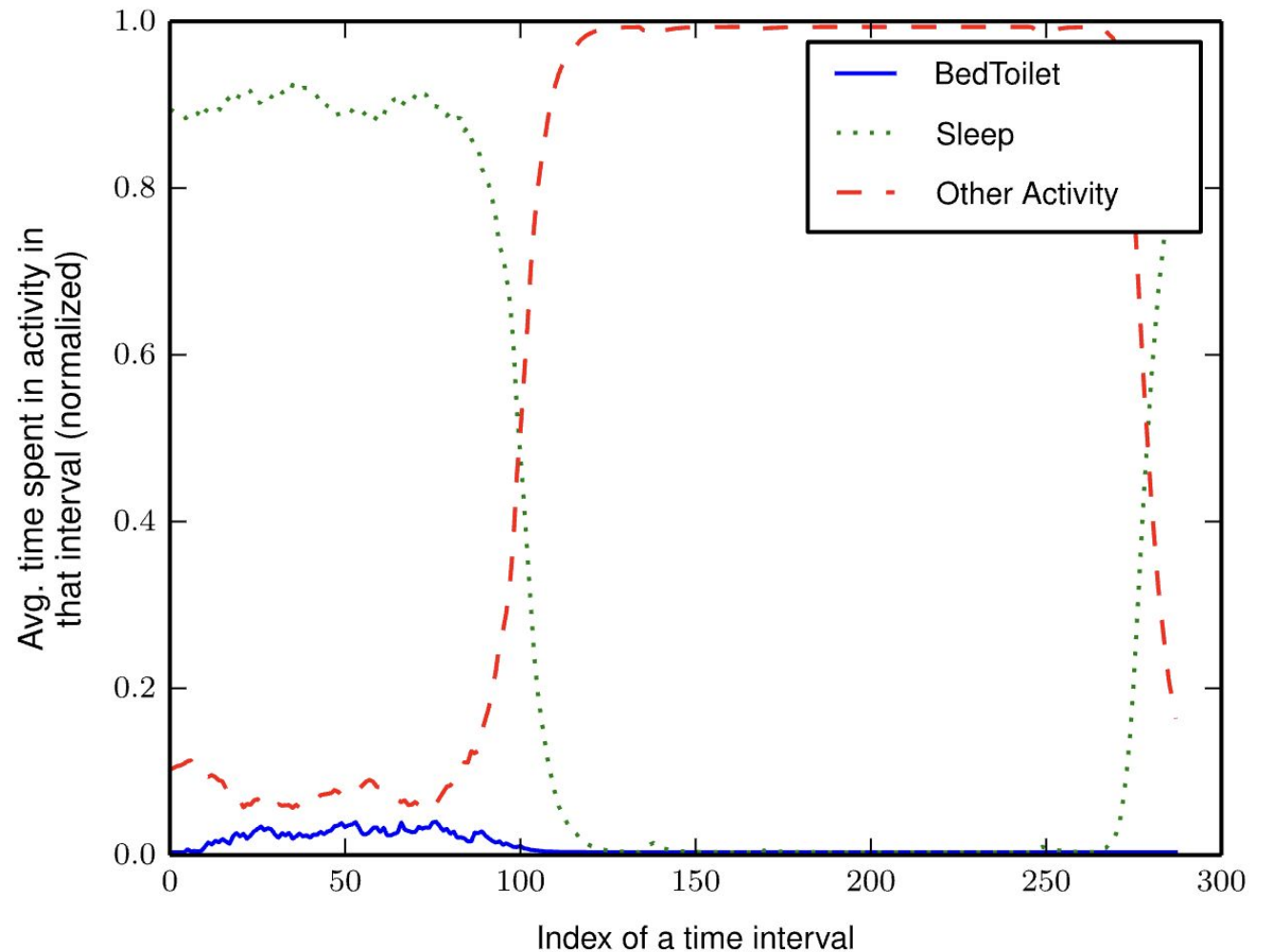


Dawadi, P. N., Cook, D. J., & Schmitter-Edgecombe, M.
(2016). Modeling patterns of activities using activity curves.
Pervasive and mobile computing, 28, 51-68.

Aggregated Activity Curve

An example of aggregated activity curve that models three different activities using three months smart home data. Aggregated activity distributions were calculated at 5 min time intervals.

In this graph, the time interval at index 0 represents 12:00 AM.



Detecting significant changes in activity curves

- Significant changes in activity curves over time may reflect clinically relevant behavioral changes
 - (e.g., showing a change in routine computed in the first year of remote monitoring compared to the routine during the second year of remote monitoring)
- To automatically detect such changes it is necessary to rely on computing the distance between probability distributions
 - e.g., between activity curves computed
- A high distance between probability distributions may indicate a behavioral change

Kullback–Leibler divergence

Distance between probability distributions can be computed as follows:

$$D_{KL}(\mathbf{D}_1 \parallel \mathbf{D}_2) = \sum_{i=1}^n d_{1,i} \log \frac{d_{1,i}}{d_{2,i}}.$$

Since this metric does not reflect differences in a symmetric way, it is necessary to use the symmetric version:

$$SD_{KL}(\mathbf{D}_1 \parallel \mathbf{D}_2) = D_{KL}(\mathbf{D}_1 \parallel \mathbf{D}_2) + D_{KL}(\mathbf{D}_2 \parallel \mathbf{D}_1).$$

Note: $\mathbf{D}_1$ and $\mathbf{D}_2$ are probability distributions (e.g., over the possible activities).
The notation $d_{1,i}$ indicates the probability value of activity i in the distribution $\mathbf{D}_1$.

Comparing activity curves

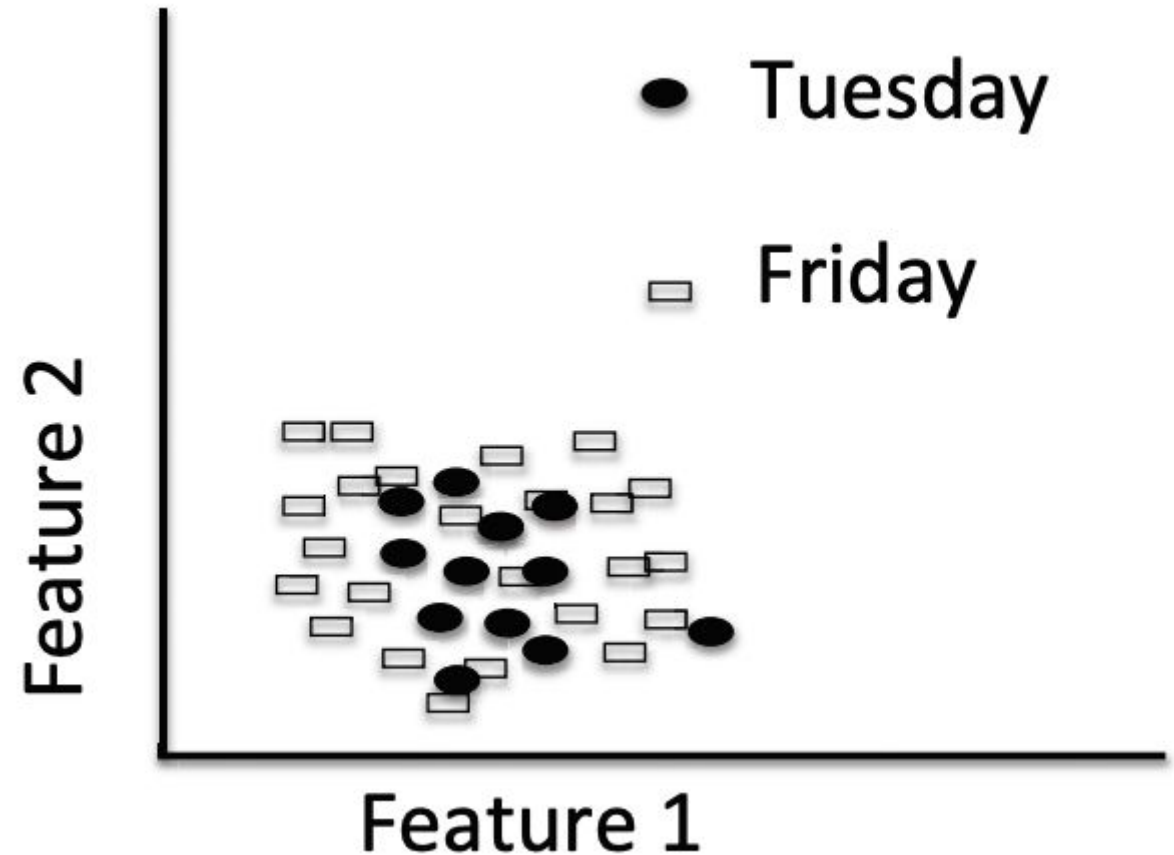
Given two aggregated activity curves, the Symmetric Divergence can be computed as the sum of divergences for all the time intervals covered by the curve.

$$SD_{KL}(C_1 \parallel C_2) = \sum_{\alpha=1}^l SD_{KL}(\mathbf{D}_{1,p} \parallel \mathbf{D}_{2,q})$$

Note: C_1 and C_2 are two activity curves (e.g., obtained in two different time periods), having a probability distribution on each considered time interval.

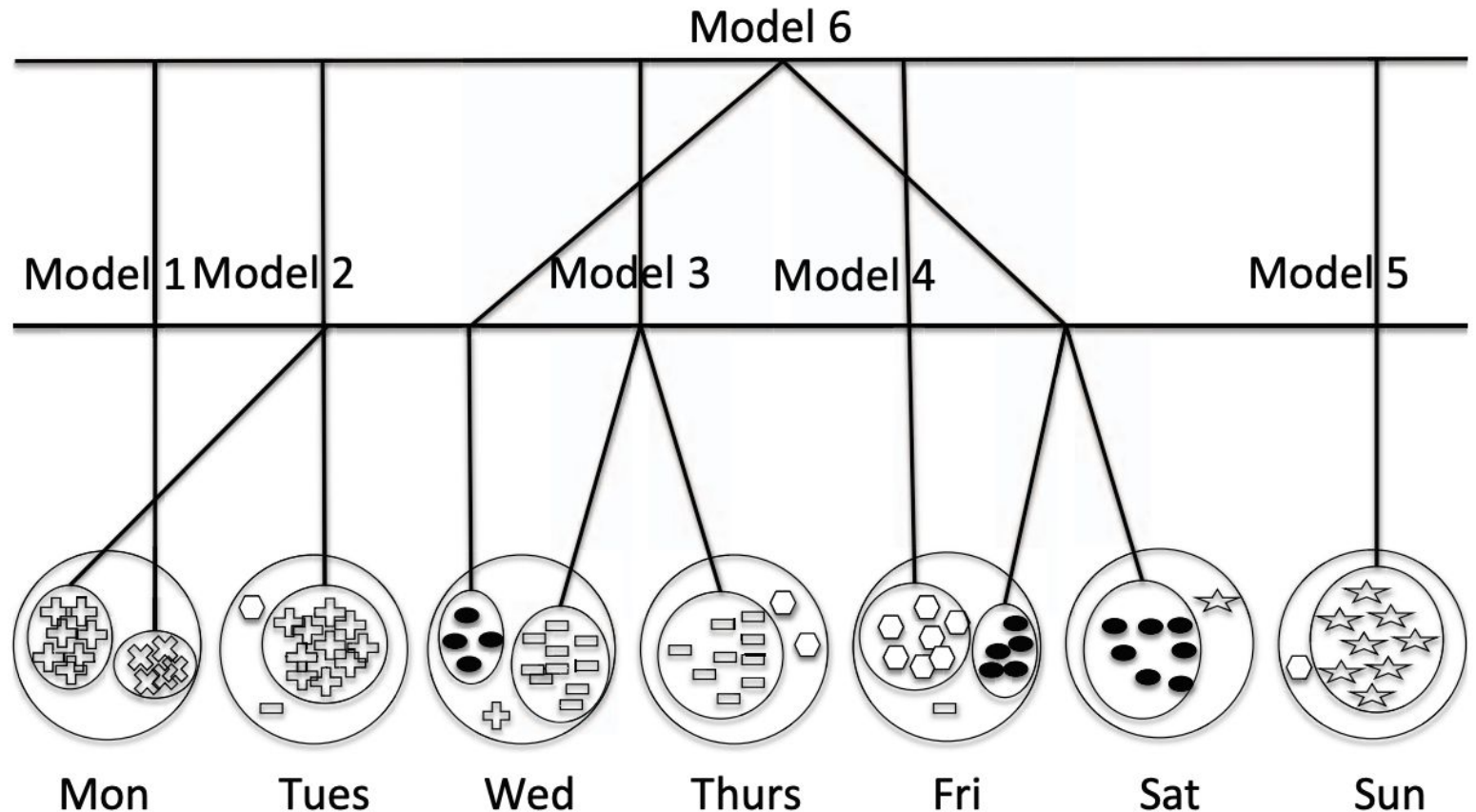
Temporal information is crucial!

- Based on the temporal context (e.g., the day of the week), the subject may perform activities with different patterns
 - e.g. having short meals during week days, longer ones in the weekends
- Hence, the “normal behavior” should take temporal context into account!



A Hierarchical Model to Model Human Behavior

- Idea:
 - create the model of normal behavior at a fine temporal granularity (e.g., day)
 - create larger behavior models by grouping them (e.g., working days, weekends, etcetera)



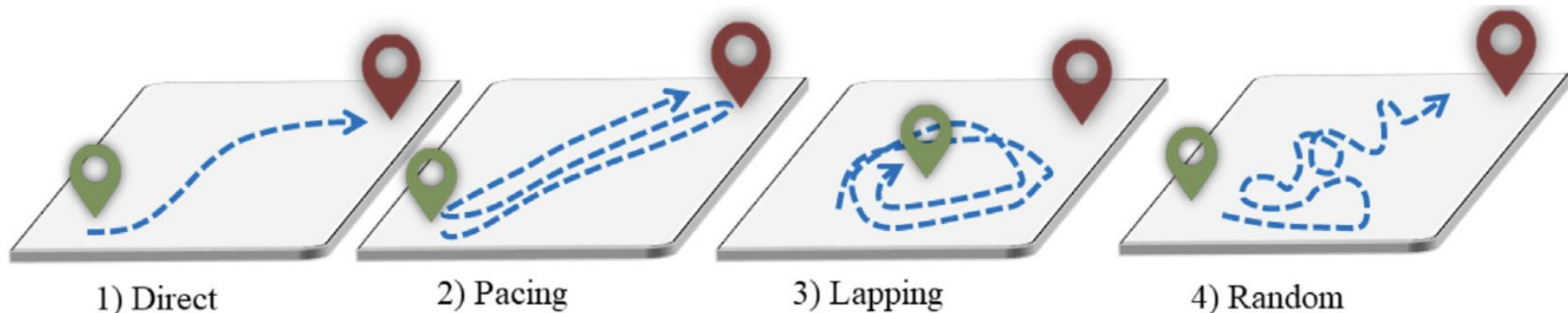
Wandering Behavior in Smart Homes

The *wandering* behavior

- In smart-homes it is possible to use localization methods to track the trajectories of subjects
- One of the symptoms of cognitive decline is also the emergence of *wandering* behaviors
 - the tendency of a subject moving around aimlessly or without a clear purpose, sometimes in a repetitive or looping pattern
 - usually characterized by *loops* in the trajectories
- Several methods have been proposed methods for analyzing human trajectories to recognize such behaviors
 - e.g., to support diagnosis, to help the subject in real-time

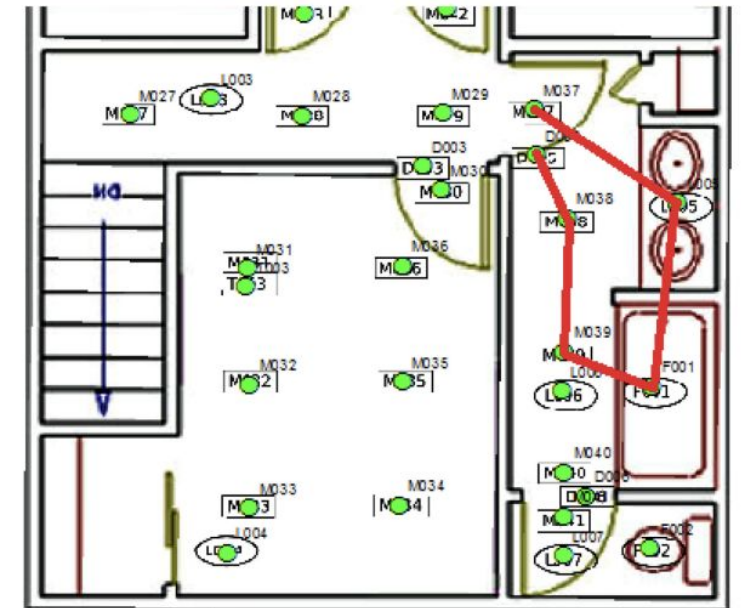
Locomotion categorization

- **Direct**: a single straightforward path from a point to a destination, which is not complicated and does not diverge significantly from the most efficient path.
- **Pacing**: at least three consecutive repeated back-and-forth movements between two locations.
- **Lapping**: a circular repeated movement that passes across at least 3 distinct points and that is repeated at least twice.
- **Random**: an aimless movement across numerous locations, which is not direct



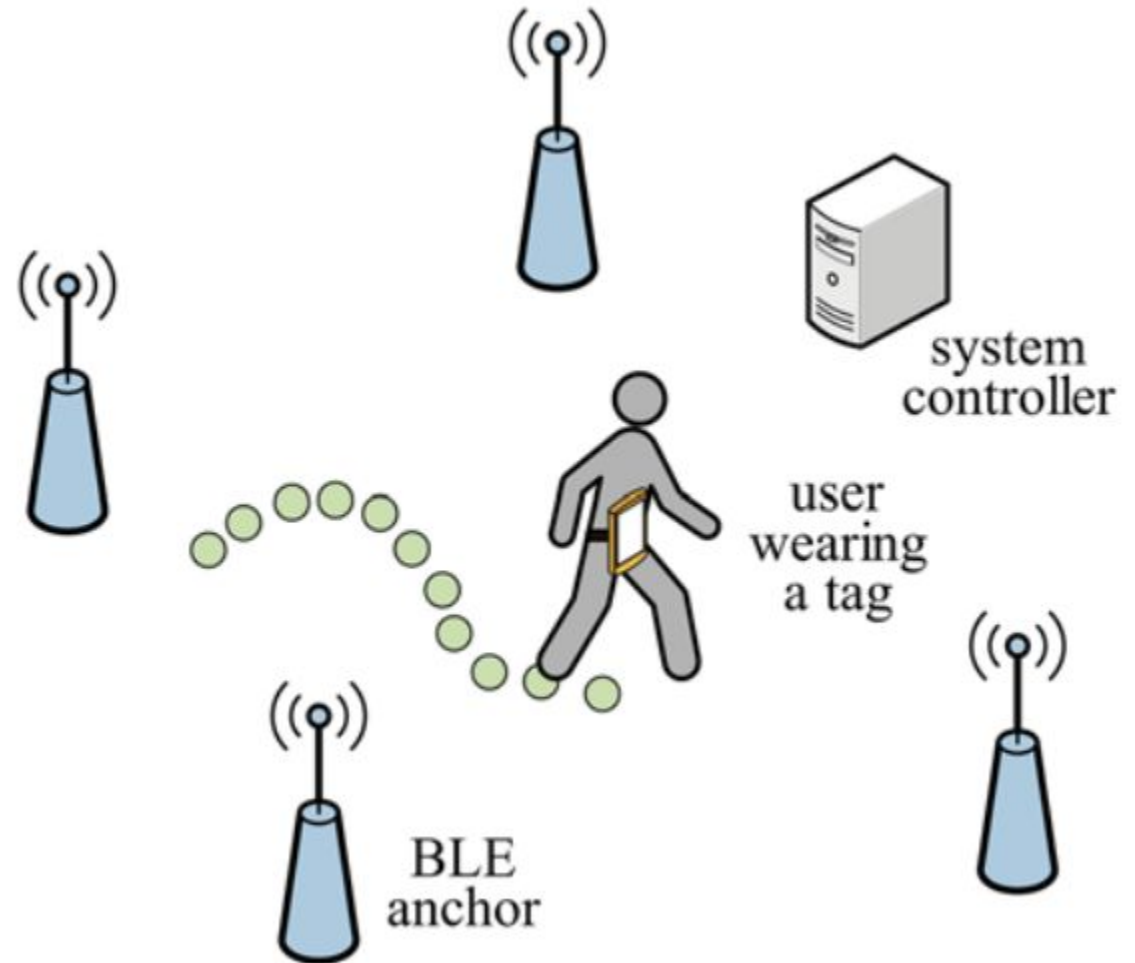
Methods based on Smart-Home Sensors

- In smart-homes, environmental sensors have a fixed positions
- Hence, it is possible to map each one of them with relative coordinates (x,y) in the home environment
- A sequence of triggered sensor events also generates a sequence of timestamped relative positions: we have an approximated trajectory



Methods based on Indoor Localization

- Wandering can also be implemented thanks to indoor localization:
 - the trajectory of the user may be estimated by periodically computing its position in the environment



Detecting wandering behaviors in smart-homes

- Idea:
 - given a sequence of user's positions, the goal is to find sub-sequences (trajectories) including a loop
 - compute the algebraic sum of points in sub-sequences
 - if the sum is close to zero (according to an approximation error ε), then a loop is detected
 - in this case, the segment contains a *wandering* behavior

Assessing the cognitive status from trajectories

- Besides finding loops, clinically relevant wandering behaviors should also consider other important features, like:
 - the area of the loop;
 - the length of the loop;
 - the relative coordinates of the centroid of the loop;
 - the time taken by the user to walk the loop
- In the literature, there are approaches based on supervised learning to classify the cognitive status from wandering behaviors
 - They are based on datasets of trajectories, labeled with the cognitive status of the subject (e.g., healthy, mild cognitive impairment, cognitively impaired)

References

- [1] Krishnan, N. C., & Cook, D. J. (2014). ***Activity recognition on streaming sensor data.*** *Pervasive and mobile computing*, 10, 138-154.
- [2] Liciotti, D., Bernardini, M., Romeo, L., & Frontoni, E. (2020). ***A sequential deep learning application for recognising human activities in smart homes.*** *Neurocomputing*, 396, 501-513.
- [3] Chen, L., & Nugent, C. (2009). ***Ontology-based activity recognition in intelligent pervasive environments.*** *International Journal of Web Information Systems*.